



ENGENHEIRO DE QUALIDADE DE SOFTWARE

Ramon Silva Cajueiro de Campos

Análise de Qualidade

Santo André

2025

1. RESUMO

O propósito deste trabalho de conclusão de curso é aplicar os conhecimentos adquiridos na formação de Engenheiro de Qualidade de Software na validação do e-commerce **EBAC Shop**. Para isso, foi desenvolvida uma estratégia de testes completa, contemplando desde o planejamento até a execução de testes manuais e automatizados em diferentes plataformas (Web, API e Mobile). Foram elaborados critérios de aceitação em linguagem Gherkin, casos de teste com técnicas como partição de equivalência e valores limite, além da definição de cenários negativos e alternativos. O projeto incluiu a implementação de automações utilizando frameworks adequados, bem como a configuração de integração contínua no GitHub Actions e a execução de testes de performance com a ferramenta K6. Dessa forma, buscou-se garantir a qualidade do produto por meio de práticas consistentes de engenharia de software, simulando a atuação real de um Quality Engineer em um time ágil.

2. SUMÁRIO

1. RESUMO.....	2
2. SUMÁRIO	3
3. INTRODUÇÃO	4
4. O PROJETO	5
4.1 Estratégia de teste	5
4.2 Critérios de aceitação.....	6
4.3 Casos de testes	6
4.4 Repositório no Github	13
4.5 Testes automatizados	13
4.6 Integração contínua	15
4.7 Testes de performance.....	15
5. CONCLUSÃO	15
6. REFERÊNCIAS BIBLIOGRÁFICAS.....	16

3. INTRODUÇÃO

O presente trabalho tem como propósito consolidar os aprendizados obtidos ao longo do curso de Engenharia de Qualidade de Software, aplicando-os em um projeto prático de validação de um sistema de comércio eletrônico. A aplicação escolhida para este estudo é o **EBAC Shop**, uma loja virtual que possibilita a exploração de funcionalidades essenciais como cadastro de usuários, login, carrinho de compras, pedidos e gerenciamento de conta. A partir desse contexto, foram elaborados critérios de aceitação, casos de teste e estratégias de validação que englobam diferentes tipos e níveis de testes. Também foram desenvolvidas automações para Web, API e Mobile, integradas em um fluxo de integração contínua. Além disso, foram executados testes de performance a fim de avaliar a robustez da aplicação sob carga. O trabalho, portanto, reflete a atuação prática de um Engenheiro de Qualidade de Software em um ambiente ágil, evidenciando não apenas a execução técnica, mas também a importância da aplicação de boas práticas e metodologias de testes para garantir a entrega de software de qualidade.

4. O PROJETO

Para este trabalho de conclusão de curso **Profissão: Engenheiro de Qualidade de software**, você deve utilizar o conhecimento adquirido ao longo do curso para elaborar uma estratégia de testes adequada para validar o e-commerce EBAC Shop (<http://lojaebac.ebaconline.art.br/>). Você deve considerar as histórias de usuário já refinadas como se você estivesse participando de um time ágil. As funcionalidades devem seguir todo o fluxo de trabalho de um *Quality Engineer* (QE), desde o planejamento até a entrega. Siga as etapas dos sub-tópicos para se orientar no trabalho.

ATENÇÃO:

- Conforme a sua estratégia, você pode executar os testes no endereço disponibilizado ou utilizando as imagens disponíveis no Docker Hub:
 - Banco de Dados: [ernestosbarbosa/lojaebacdb](https://hub.docker.com/r/ernestosbarbosa/lojaebacdb)
 - Loja EBAC: [ernestosbarbosa/lojaebac](https://hub.docker.com/r/ernestosbarbosa/lojaebac)
- Comandos para subir os containers:

```
docker network create --attachable ebac-network  
  
docker run -d --name wp_db -p 3306:3306 --network ebac-network ernestosbarbosa/lojaebacdb:latest  
  
docker run -d --name wp -p 80:80 --network ebac-network ernestosbarbosa/lojaebac:latest
```

Após subir os containers a loja estará em <http://localhost:80>

- Como este trabalho complementa o que criou em seu Trabalho de Consolidação (Módulo 19), você pode utilizá-lo como base para o seu Trabalho de Conclusão.

4.1 Estratégia de teste

- Faça uma estratégia de testes em um mapa mental, seguindo algumas diretrizes como objetivos, papéis e responsabilidades, fases de testes, padrões, tipos de testes, técnicas de testes, ambientes, ferramentas, abordagem (manual ou automatizado), framework ou ferramenta usados, plataformas (web, api, mobile), etc.;
- Referência: Módulo 5
- Após fazer sua estratégia de teste, tire um print e cole aqui:



Presented with xmind AI

4.2 Critérios de aceitação

- Considere as histórias de usuário:
 - [US-0001] – Adicionar item ao carrinho
 - [US-0002] – Login na plataforma
 - [US-0003] – API de cupons
- Para cada uma delas crie pelo menos 4 critérios de aceitação usando a linguagem Gherkin;
- Crie histórias de usuário para as funcionalidades:
 - Catálogo de Produtos
 - Painel Minha Conta
 - Meus Pedidos
 - Endereços
 - Detalhes da Conta
- Referência: Módulo 8

4.3 Casos de testes

- Crie pelo menos 4 casos de testes para cada história de usuário, sempre que possível, usando as técnicas de testes (partição de equivalência, valor limite, tabela de decisão etc.).
- Considere sempre o caminho feliz (fluxo principal) e o caminho alternativo e negativo (fluxo alternativo). Exemplo de cenário negativo: “Ao preencher com usuário e senha inválidos deve exibir uma mensagem de alerta...”

- Identifique quais os casos de teste serão automatizados, sendo ao menos 1 caminho feliz e 1 caminho alternativo.
- Referência: Módulos 4 e 5

História 1 – Catálogo de Produtos

CT001 – Visualizar catálogo com filtro de categoria

Descrição:	Validar exibição do catálogo ao aplicar filtro por categoria
Pré-condições:	Usuário no site da EBAC-SHOP
Massa de dados / Parâmetros:	Categoria "Eletrônicos"
Prioridade:	Alta

Ação	Resultado esperado
1. Acessar página de catálogo	
2. Selecionar filtro "Eletrônicos"	Apenas produtos da categoria "Eletrônicos" exibidos

CT002 – Pesquisa por produto inexistente

Descrição:	Validar mensagem ao pesquisar por item não cadastrado
Pré-condições:	Usuário no site da EBAC-SHOP
Massa de dados / Parâmetros:	Termo de busca "XPhone Z Pro"
Prioridade:	Média

Ação	Resultado esperado
1. Acessar barra de pesquisa	
2. Digitar "XPhone Z Pro" e confirmar	Exibir mensagem "Nenhum produto encontrado"

CT003 – Filtro por preço mínimo no limite inferior

Descrição:	Validar filtro de preço com valor mínimo aceito
Pré-condições:	Usuário no site da EBAC-SHOP
Massa de dados / Parâmetros:	Preço mínimo = R\$ 0,00

Prioridade:	Baixa
-------------	-------

Ação	Resultado esperado
1. Abrir filtro de preço	
2. Inserir mínimo R\$ 0,00 e aplicar	Listar todos os produtos disponíveis no catálogo

CT004 – Aplicar múltiplos filtros simultaneamente

Descrição:	Validar funcionamento da busca com múltiplos filtros
Pré-condições:	Usuário no site da EBAC-SHOP
Massa de dados / Parâmetros:	Categoria "Roupas", marca "EBAC Wear", preço até R\$ 200
Prioridade:	Média

Ação	Resultado esperado
1. Acessar página de catálogo	
2. Aplicar filtros especificados	Exibir apenas produtos que atendam a todos os filtros

História 2 – Painel Minha Conta

CT005 – Acesso ao painel após login válido

Descrição:	Validar acesso ao painel “Minha Conta” com credenciais válidas
Pré-condições:	Usuário cadastrado com conta ativa
Massa de dados / Parâmetros:	Usuário: cliente@ebac.com, Senha: 123456
Prioridade:	Alta

Ação	Resultado esperado
1. Acessar a página de login	
2. Inserir e-mail e senha válidos e clicar em “Entrar”	Redirecionamento para o painel “Minha Conta” com opções disponíveis

CT006 – Bloqueio de acesso sem login

Descrição:	Validar bloqueio de acesso a um usuário não cadastrado
Pré-condições:	Usuário não logado

Massa de dados / Parâmetros:	Usuário: cliente@ebac.com, Senha: 000000
Prioridade:	Alta

Ação	Resultado esperado
1. Acessar a página de login	
2. Inserir e-mail e senha inválidos e clicar em “Entrar”	Exibição e mensagem “usuário ou senha inválidos”

CT007 – Sessão expirada

Descrição:	Validar redirecionamento ao login quando sessão expira
Pré-condições:	Usuário logado, tempo de sessão configurado
Massa de dados / Parâmetros:	Tempo limite = 30 min inatividade
Prioridade:	Média

Ação	Resultado esperado
1. Logar no sistema	
2. Aguardar 30 min sem interação	
3. Clicar em qualquer link do painel	Redirecionamento para login

CT008 – Bloqueio de acesso sem login

Descrição:	Validar que usuário não logado é redirecionado para login ao tentar acessar “Minha Conta”
Pré-condições:	Usuário não logado
Massa de dados / Parâmetros:	URL /minha-conta
Prioridade:	Alta

Ação	Resultado esperado
1. Digitar URL /minha-conta diretamente no navegador	Sistema redireciona para tela de login

História 3 – Meus Pedidos

CT009 – Visualizar histórico de pedidos

Descrição:	Validar listagem de pedidos para cliente logado com pedidos existentes
Pré-condições:	Cliente logado com histórico de compras
Massa de dados / Parâmetros:	Usuário: cliente@ebac.com, Senha: 123456

Prioridade:	Alta
-------------	------

Ação	Resultado esperado
1. Logar no sistema	
2. Acessar opção “Meus Pedidos”	Lista de pedidos exibida com informações corretas

CT010 – Histórico vazio

Descrição:	Validar mensagem quando cliente não possui pedidos
Pré-condições:	Cliente logado sem histórico de compras
Massa de dados / Parâmetros:	Usuário: novocliente@ebac.com, Senha: 123456
Prioridade:	Média

Ação	Resultado esperado
1. Logar no sistema	
2. Acessar opção “Meus Pedidos”	Mensagem “Nenhum pedido encontrado” exibida

CT011 – Ordenação por data

Descrição:	Validar ordenação decrescente de pedidos por data
Pré-condições:	Cliente logado com múltiplos pedidos
Massa de dados / Parâmetros:	Lista de pedidos com datas diferentes
Prioridade:	Baixa

Ação	Resultado esperado
1. Logar no sistema	
2. Acessar “Meus Pedidos”	Pedidos exibidos da data mais recente para a mais antiga

CT012 – Filtrar por status

Descrição:	Validar filtro por status de pedido
Pré-condições:	Cliente com pedidos em diferentes status
Massa de dados / Parâmetros:	Status: “Entregue”
Prioridade:	Média

Ação	Resultado esperado
1. Logar no sistema	
2. Acessar “Meus Pedidos”	

3. Aplicar filtro “Entregue”	Apenas pedidos entregues exibidos
------------------------------	-----------------------------------

História 4 – Endereços

CT013 – Adicionar endereço válido

Descrição:	Validar adição de novo endereço com dados corretos
Pré-condições:	Cliente logado
Massa de dados / Parâmetros:	Rua “Av. Paulista, 1000”, CEP “01310-000”, Cidade “São Paulo”, Estado “SP”
Prioridade:	Alta

Ação	Resultado esperado
1. Acessar “Minha Conta”	
2. Clicar em “Adicionar novo endereço”	
3. Preencher todos os campos obrigatórios corretamente	Endereço salvo e listado com sucesso

CT014 – CEP inválido

Descrição:	Validar mensagem de erro ao inserir CEP inválido
Pré-condições:	Cliente logado
Massa de dados / Parâmetros:	CEP “00000-000”
Prioridade:	Alta

Ação	Resultado esperado
1. Acessar “Minha Conta”	
2. Clicar em “Adicionar novo endereço”	
3. Preencher dados corretos, mas inserir CEP inválido	Mensagem “CEP inválido” exibida e endereço não salvo

CT015 – Editar endereço existente

Descrição:	Validar atualização de endereço já cadastrado
Pré-condições:	Cliente logado e com endereço salvo
Massa de dados / Parâmetros:	Alterar número da residência
Prioridade:	Média

Ação	Resultado esperado
1. Acessar “Minha Conta”	

2. Clicar em “Editar endereço”	
3. Alterar campo “Número” e salvar	Mensagem “Endereço atualizado com sucesso” e endereço salvo

CT016 – Campo obrigatório não preenchido

Descrição:	Validar mensagem de erro ao deixar campo obrigatório vazio
Pré-condições:	Cliente logado
Massa de dados / Parâmetros:	Campo “Rua” vazio
Prioridade:	Média

Ação	Resultado esperado
1. Acessar “Minha Conta”	
2. Clicar em “Adicionar novo endereço”	
3. Deixar campo “Rua” vazio e salvar	Mensagem “Campo obrigatório” exibida e endereço não salvo

História 5 – Detalhes da Conta

CT017 – Atualizar dados pessoais válidos

Descrição:	Validar atualização de nome, e-mail e telefone corretos
Pré-condições:	Cliente logado
Massa de dados / Parâmetros:	Nome “Maria Silva”, E-mail “maria@ebac.com”, Telefone “11987654321”
Prioridade:	Alta

Ação	Resultado esperado
1. Acessar “Minha Conta”	
2. Atualizar todos os campos com dados válidos e salvar	Mensagem “Dados atualizados com sucesso”

CT018 – E-mail inválido

Descrição:	Validar mensagem de erro ao inserir e-mail em formato incorreto
Pré-condições:	Cliente logado
Massa de dados / Parâmetros:	E-mail “maria@@ebac.com
Prioridade:	Alta

Ação	Resultado esperado
-------------	---------------------------

1. Acessar “Minha Conta”	
2. Inserir e-mail inválido e salvar	Mensagem “Formato de e-mail inválido” exibida

CT019 – Alterar apenas telefone

Descrição:	Validar atualização de apenas um campo mantendo os outros inalterados
Pré-condições:	Cliente logado com dados cadastrados
Massa de dados / Parâmetros:	Telefone “11912345678”
Prioridade:	Média

Ação	Resultado esperado
1. Acessar “Minha Conta”	
2. Alterar apenas telefone e salvar	Telefone atualizado, demais campos inalterados

CT020 – Todos os campos vazios

Descrição:	Validar erro ao tentar salvar sem preencher nenhum campo
Pré-condições:	Cliente logado
Massa de dados / Parâmetros:	Campos vazios
Prioridade:	Média

Ação	Resultado esperado
1. Acessar “Minha Conta”	
2. Clicar em salvar sem preencher nada	Mensagem “Preencha os campos obrigatórios” exibida

4.4 Repositório no Github

- Crie um repositório no github com o nome TCC-EBAC-QE;
- Deixe o repositório publico até a análise dos tutores;
- Neste repositório você deve subir este arquivo e todos os código fontes das automações que criar.
- Referência: Módulo 10
- Link do repositório: <https://github.com/RCajueiro/TCC-EBAC-QE.git>

4.5 Testes automatizados

4.5.1 Automação de UI

- Crie um projeto de automação WEB com o framework e a linguagem que preferir
- Justifique a sua escolha através de um comparativo entre ao menos 3 opções de ferramentas e linguagem.
- Crie uma pasta chamada UI para os testes WEB dos casos de teste que forem automatizados
- Utilize ao menos um *Testing Pattern* (à sua escolha) na implementação dos testes.

A escolha do Cypress aliado ao JavaScript para a automação dos testes de UI se justifica pela sua simplicidade de configuração, curva de aprendizado reduzida e integração nativa com o ecossistema web. Diferente de outras ferramentas, como Selenium e Playwright, o Cypress executa os testes diretamente no mesmo ciclo de execução da aplicação, proporcionando maior rapidez e confiabilidade na detecção de falhas. Além disso, oferece recursos integrados como debugging visual e geração de relatórios, eliminando a necessidade de diversas bibliotecas adicionais. O uso de JavaScript reforça a escolha, pois é a linguagem padrão do desenvolvimento web, permitindo maior aderência entre os testes automatizados e a aplicação sob teste, além de facilitar a colaboração entre desenvolvedores e testadores dentro de um time ágil.

4.5.2 Automação de API

- Crie uma pasta chamada API para os testes de API dos casos de teste que forem automatizados
- Você deve utilizar a ferramenta Supertest para criar seus testes de API
- Não esqueça de validar os contratos! 😊

4.5.3 Automação Mobile

- Considere para os APPs apenas a funcionalidade de Catálogo de Produtos
- Você pode encontrar os APPs em:
 - *Android*: <https://github.com/EBAC-QE/testes-mobile-ebac-shop/tree/main/app/android>
 - *iOS*: <https://github.com/EBAC-QE/testes-mobile-ebac-shop/tree/ios-tests/app/ios>
- Crie uma pasta chamada Mobile para os testes em aplicativos dos casos de teste que forem automatizados
- Utilize ao menos um *Testing Pattern* (à sua escolha) na implementação dos testes.
- Você deve implementar testes para ao menos uma das plataformas Mobile (*Android* ou *iOS*)

- Observações:

- Considere todas as boas práticas aprendidas até aqui
 - Não esqueça de implementar a geração de relatórios
- Referência: Módulos 11, 12, 14, 16, 17, 22, 23, 24, 29 e 30

4.6 Integração contínua

- Execute os testes automatizados em integração contínua utilizando o Github Actions
- Referência: Módulo 26

4.7 Testes de performance

- Usando o K6, implemente um teste de performance em ao menos 2 casos de testes
- Referência: Módulo 28

- Configurações do teste de performance:

-Usuários virtuais: 20
 -Tempo de execução: 2 minutos
 -RampUp: 20 segundos
 -Massa de dados: Usuário / senha:

user1_ebac / psw!ebac@test
 user2_ebac / psw!ebac@test
 user3_ebac / psw!ebac@test
 user4_ebac / psw!ebac@test
 user5_ebac / psw!ebac@test

☐ Nome de usuário	Nome	E-mail	Função
☐  user1_ebac	—	user1_ebac@ebac.com	Assinante
☐  user2_ebac	—	user2_ebac@ebac.com	Assinante
☐  user3_ebac	—	user3_ebac@ebac.com	Assinante
☐  user4_ebac	—	user4_ebac@ebac.com	Assinante
☐  user5_ebac	—	user5_ebac@ebac.com	Assinante

5. CONCLUSÃO

Por fim, a realização deste trabalho permitiu aplicar de forma prática os conceitos estudados ao longo do curso, proporcionando uma visão abrangente do papel do Engenheiro de Qualidade de Software no ciclo de desenvolvimento. Foi possível compreender a importância do planejamento estruturado, da definição de critérios claros e da execução de testes em diferentes níveis e plataformas para assegurar a qualidade do produto. A experiência prática com ferramentas de automação, integração contínua e testes de performance trouxe uma compreensão mais profunda sobre como essas práticas fortalecem a confiabilidade de um sistema em produção. Além disso, o contato com técnicas como partição de equivalência, valores limite e cenários negativos reforçou a relevância de uma abordagem crítica e detalhista no processo de validação. Como lição principal, este trabalho demonstrou que a qualidade de software vai além da execução de testes: envolve planejamento, estratégia, automação e monitoramento contínuo. Esses aprendizados certamente poderão ser aplicados na vida profissional, contribuindo para o desenvolvimento de soluções mais seguras, escaláveis e alinhadas às necessidades do usuário.

6. REFERÊNCIAS BIBLIOGRÁFICAS

APPIUM. *Appium Documentation*. Disponível em: <https://appium.io/docs/en/latest/>. Acesso em: 22 ago. 2025.

CYPRESS. *Cypress Documentation*. Disponível em: <https://docs.cypress.io/app/get-started/why-cypress>. Acesso em: 22 ago. 2025.

GRAFANA LABS. *K6 Documentation*. Disponível em: <https://grafana.com/docs/k6/latest/>. Acesso em: 22 ago. 2025.

MICROSOFT. *GitHub Actions Documentation*. Disponível em: <https://docs.github.com/pt/actions>. Acesso em: 22 ago. 2025.

NODE.JS FOUNDATION. *Supertest – Super Agent driven library for testing HTTP servers*. Disponível em: <https://github.com/ladjs/supertest>. Acesso em: 22 ago. 2025.

WEBDRIVERIO. *WebdriverIO Documentation*. Disponível em: <https://webdriver.io/docs/gettingstarted>. Acesso em: 22 ago. 2025.